Intelligent DevOps Automation Platform

AI-Assisted Failure Analysis & Automated Recovery

Project Proposal | DevOps-first | AWS EC2

Compact project overview | proposed design, not completed implementation

Project Idea

- A web-based platform to automate software build, testing, deployment, monitoring and recovery.
- Jenkins is the central CI/CD engine; AWS EC2 hosts the application and supporting services.
- AI is an add-on that helps diagnose application failures—not the main project.

Problem Statement

- Build and deployment steps are often manual or spread across tools.
- Pipeline failures and application incidents are difficult to track together.
- Monitoring reveals symptoms, but engineers still investigate root causes manually.
- Recovery and rollback can be slow and inconsistent.

Objectives

- Automate CI/CD using Jenkins and GitHub.
- Provision cloud infrastructure using Terraform.
- Containerize applications with Docker and deploy on AWS EC2/Kubernetes.
- Monitor health, metrics and logs using Prometheus, Grafana and Loki.
- Add AI-assisted root cause analysis and safe rollback workflows.
- Provide a single interactive website to manage and observe the workflow.

System Architecture

- GitHub → Jenkins → Build / Test / Scan → Docker Image → Registry
- → AWS EC2 / Kubernetes Deployment → Application
- Application → Prometheus + Grafana + Loki → Incident Collector
- Incident Collector → RCA Engine → FastAPI → React Dashboard
- Terraform provisions infrastructure; Jenkins executes delivery pipelines.

Website Modules

- Dashboard: pipeline status, releases, service health and incidents.
- Pipeline: Jenkins stages, logs, build history and test reports.
- Deployment: environments, image versions, rollout and rollback.
- Monitoring: CPU, memory, latency, error rate and availability.
- Logs & Incidents: filtered logs, alerts, probable cause and evidence.
- Recovery: approved rollback/restart and post-recovery health check.

Jenkins CI/CD Pipeline

- 1. GitHub push / webhook triggers Jenkins.
- 2. Checkout code and install dependencies.
- 3. Run unit tests and code-quality/security checks.
- 4. Build and scan Docker image; push to registry.
- 5. Deploy to EC2/Kubernetes and run smoke tests.
- 6. Send pipeline/deployment status to the website.
- Use Jenkins credentials; approval gate for production-like releases.

Cloud, Containers & Monitoring

- AWS EC2: host Jenkins, application and/or monitoring components (separate where feasible).
- Terraform: repeatable infrastructure provisioning.
- Docker: consistent application packaging.
- Kubernetes: rollout, health probes and scaling (can be added after Docker baseline).
- Prometheus: metrics; Grafana: dashboards; Loki: centralized logs.
- Security: least-privilege IAM, restricted ports, TLS and secret management.

AI Root Cause Analysis

- Use RCAEval as a research benchmark after reviewing its schema and licensing.
- Process incident-window metrics/log patterns and service dependency context.
- Start with rule-based baseline; compare with ML classifier/ranker.
- Candidate models: Isolation Forest for anomaly detection; XGBoost/Random Forest for cause prediction.
- Show top probable cause, evidence, confidence and suggested checks.
- AI recommendations remain advisory; operator approves recovery.

Implementation Roadmap

- 1. Build demo application and GitHub repository.
- 2. Set up AWS EC2 and Terraform.
- 3. Configure Jenkins CI pipeline and Docker build.
- 4. Automate deployment and smoke testing.
- 5. Add Prometheus, Grafana and log collection.
- 6. Build React + FastAPI website integration.
- 7. Train/evaluate RCA model and integrate incidents.
- 8. Add approval-based rollback; test and document demo.

Expected Final Output

- A working, interactive DevOps website.
- Automated Jenkins build-test-deploy pipeline.
- Terraform-managed AWS infrastructure and container deployment.
- Monitoring dashboards, centralized logs and incident view.
- AI-assisted root cause analysis with explainable evidence.
- Controlled rollback/recovery demonstration and project report.

Research Focus & Conclusion

- Core engineering contribution: end-to-end DevOps automation and incident workflow.
- Potential research angle: telemetry-informed, explainable root-cause ranking connected to safe recovery.
- Research novelty must be confirmed through literature review; benchmark results and live-demo results should be reported separately.
- Final emphasis: DevOps automation first; AI is an intelligent enhancement.